

Professional Training in Cybersecurity I

Module 5 — Capstone

Capstone Project Deliverable — Slot B2

Mohamed Gebril

May 13, 2026

Project packet

This packet is for Slot B2 only. It contains the project topic, shared capstone requirements, confirmed dates, demo rules, safety rules, and submission checklist. Team member names are intentionally not included.

Confirmed schedule

Code, Docker setup, README, sample logs, and presentation slides are due **Friday, May 29, 2026**. Demo and presentation day is **Monday, June 1, 2026**. The final technical report is due **Thursday, June 4, 2026**.

Docker lab only

Use only the included `docker-compose.yml` lab environment for development, testing, and demonstration. Do not run attacks against real systems.

Table of Contents

1 Slot B2 — UDP Flooder + UDP Rate Anomaly Detector	3
1.1 Background	3
1.2 Attack Tool: UDP Flooder	3
1.3 Defense Tool: UDP Rate Anomaly Detector	4
2 Shared Capstone Requirements	5
3 Confirmed Dates	5
4 Deliverables and Weights	5
5 Timeline	6
6 Demo and Presentation Day Format	6
7 Lab Safety Rules	6
8 Submission Checklist	7

1 Slot B2 — UDP Flooder + UDP Rate Anomaly Detector

1.1 Background

UDP (User Datagram Protocol) is connectionless and stateless, making it trivial to generate at high volume. UDP floods exhaust target bandwidth and CPU, causing denial of service. A sophisticated flooder avoids simple rate thresholds by varying its traffic pattern: bursting, pausing, then bursting again. The defense must use a rolling-average approach to catch burst patterns that fall below per-second thresholds on average.

1.2 Attack Tool: UDP Flooder

Goal

Build a configurable multi-threaded UDP flooder with three attack modes and deliberate pattern evasion.

Minimum Feature Set

- Multi-threaded packet generation with configurable thread count and per-thread send rate.
- Two required modes (selectable via `--mode`):
 - `flood`: high-volume UDP packets to a single port.
 - `spray`: random destination port per packet (port range configurable).
- **Evasion technique**: randomised source port per packet, randomised payload content (no fixed-length signature), and a `--burst` mode that alternates high-rate bursts with silent pauses (configurable burst duration and pause duration).
- Structured JSON stats log: packets sent, bytes sent, measured send rate, target, mode, timestamp.
- All parameters via `argparse`: `--target`, `--port`, `--mode`, `--rate`, `--payload-size`, `--threads`, `--duration`, `--burst-on`, `--burst-off`, `--log`.

Docker Environment

3 containers: `attacker`, `target` (running a UDP echo service on a known port), `detector` (co-located with the target's network namespace — see `docker-compose.yml` in `docker/B2/`).

Shared network namespace

Some containers in this lab (typically the detector) share another service's network namespace via Docker's `network_mode: "service:..."`. These containers will not appear with their own IP address when you run `docker compose ps`; this is intentional and required for the detector to see unicast traffic that a Docker bridge would otherwise hide.

Bonus Enhancement (optional, extra credit)

Add a third `amplify` mode: send small payloads to an open UDP service that returns a larger response (DNS or NTP-style stub responder). Demonstrate the amplification factor (bytes-out / bytes-in) in your report. This requires implementing a small responder container, which is out of required scope.

1.3 Defense Tool: UDP Rate Anomaly Detector

Goal

Detect UDP flood and spray attacks in real time, including burst-pattern floods that pause between waves and would evade a naive per-second rate threshold.

Minimum Feature Set

- Scapy-based sniffer capturing UDP traffic on a monitored interface.
- Per-source-IP tracking with three independent detection patterns:
 - **Flood:** UDP packet rate from one source to one destination port exceeds a configurable threshold.
 - **Spray:** number of unique destination ports targeted by one source exceeds a configurable threshold within a sliding window.
 - **Burst:** short-window peak rate exceeds a multiple of the rolling average rate (configurable multiplier); this catches burst-then-pause traffic that looks low on average.
- Must catch burst-mode floods: demonstrate that flood and spray thresholds alone miss the burst pattern, but the burst detector catches it.
- Structured JSON alert log: source IP, attack pattern type, measured value, threshold exceeded, timestamp.
- All parameters via `argparse`: `--interface`, `--flood-threshold`, `--port-threshold`, `--burst-multiplier`, `--window`, `--whitelist`, `--log`.

Integration Point

Show in the demo: (1) flooder in burst mode — flood and spray detectors do not fire consistently; burst detector fires; (2) flooder in spray mode — spray detector fires; (3) explain the rolling-average implementation.

Hint

For burst detection, keep a deque of (timestamp, packet_count) tuples. For the short window (e.g. 2 s), sum recent counts and compare to the rolling average over the long window (e.g. 30 s). If short-window rate > multiplier × long-window rate, it is a burst.

Forbidden shortcuts

Implement UDP packet generation using Python's `socket` module (`SOCK_DGRAM`). Do not use `hping3`, `scapy.send()` with a rate loop is acceptable, but raw socket is preferred.

2 Shared Capstone Requirements

Universal requirements apply

Both the attack tool and the defense tool must satisfy all eight universal technical requirements. Tools that skip any requirement are capped at 50% of the code mark.

1. **Multi-threading:** use it where it provides real concurrency benefit; justify in the report if it is not used.
2. **argparse CLI:** all important values must be command-line arguments, with no hard-coded target values.
3. **JSON logging to file:** outputs and alerts must be timestamped and structured.
4. **Docker network:** the project must run in an isolated Docker lab, not against real targets.
5. **Evasion technique:** the attack tool must include the required evasion behavior from the topic card.
6. **Integration:** the defense tool must detect the team's own attack tool in real time.
7. **Robustness:** thresholds and inputs must be configurable, and the detector should avoid false positives on a clean baseline.
8. **Equal depth:** attack and defense tools must be developed with comparable effort and technical depth.

3 Confirmed Dates

Date	Milestone
Friday, May 29, 2026	Code, Docker setup, README, sample logs, and presentation slides due.
Monday, June 1, 2026	Demo and presentation day.
Thursday, June 4, 2026	Final technical report due.

4 Deliverables and Weights

Deliverable	What it includes	Weight
Live Demo	Attack and defense running live, with Q&A	35%
Technical Report	8–15 pages, penetration-test style, with evidence	25%
Source Code	Working code, README, and Docker setup	25%
Presentation	Slides, with all members speaking	15%
Peer assessment multiplier, applied individually		±10%

5 Timeline

Period	Goal
Days 1–2	Read the topic card, design the architecture, and set up Docker.
Days 3–5	Build and test the attack tool.
Days 6–8	Build and test the defense tool.
Checkpoint	Show Docker running and at least one tool producing output.
Before Friday, May 29	Integrate evasion, tune detection thresholds, validate clean baseline behavior, finalize slides, and rehearse end-to-end at least three times.
Friday, May 29, 2026	Submit code, Docker setup, README, sample logs, and presentation slides.
Monday, June 1, 2026	Demo and presentation day.
Thursday, June 4, 2026	Submit the final technical report, including live demo results.

6 Demo and Presentation Day Format

Date: Monday, June 1, 2026

Each team has **20 minutes**:

- **15 min:** presentation and live demo.
- **5 min:** Q&A.

During the demo, the team must show:

- `docker-compose up`: containers running.
- Defense tool listening first.
- Attack tool executing.
- Defense tool detecting in real time.
- The evasion technique specifically demonstrated.

7 Lab Safety Rules

Violation = zero grade for the entire project

All project activity must remain inside the assigned Docker lab environment.

- All attacks must run **inside Docker only**.
- Do not target your host machine, classmates, or the internet.
- Use an isolated bridge network with no internet access for attacker containers.
- Raw sockets and promiscuous mode must run **inside the container**.

8 Submission Checklist

Before Demo and Presentation Day: Friday, May 29, 2026

Submit the code package as a ZIP file or repository containing:

- Attack tool .py file.
- Defense tool .py file.
- `docker-compose.yml` and any Dockerfiles used by the team.
- `README.md`.
- Sample JSON log files.
- Presentation slides as PDF or PPTX.

After Demo and Presentation Day: Thursday, June 4, 2026

Submit the final documents:

- Technical report as PDF, including live demo results.
- Peer assessment form, submitted individually and privately after the demo.

Submission channel

All submissions go through the course portal.